# Zero Day Engineering

## research & consulting

Zero Day Engineering LLC
http://zerodayengineering.com

# HYPERVISOR VULNERABILITY RESEARCH

## TRAINING OVERVIEW

This professional deep technical training offers a systematical foundation for low-level security research of arbitrary hypervisor systems, taught from an offensive security research perspective by an acclaimed hacker specialized in the subject. Key objectives: threat modeling, vulnerability discovery and prototyping.

Conceptually, course materials are structured around the hypervisor master threat model, which distinguishes 4 major subsystems / attack surfaces in any modern hypervisor; each day is focused on one. We expose students to all the essential theory required to attack a particular hypervisor subsystem, as well as fundamental practical skills, attack vectors and previously discovered security vulnerabilities in each subsystem. Furthermore, a hacker's review of four popular hypervisor implementations is included.

All exercises and labs in this training course are based on real life tasks, and target essential vulnerability research skills. Training materials are updated and refreshed prior to each live class. 32 hours of study.

## AUDIENCE

This training is primarily intended for professional security researchers and virtualization systems engineers.

Materials of this training constitute an essential base of knowledge for an effective zero day vulnerability researcher targeting any hypervisor implementation.

Training approach and materials are suitable for complete beginners with application security and hypervisors.

## PREREQUISITES

Essential:

- C/C++ (reading level);
- Linux command line and build environment.

Recommended:

- x86_64 assembly language;
- some experience with vulnerability research;
- basic OS and hardware design concepts.

## PLAN

Day 1. Foundation.
Virtualization technology fundamentals. Virtual machine guest services. Oracle VirtualBox.

Day 2. Virtual hardware.
Virtual devices subsystem. OS & hardware design theory. VMware Workstation.

Day 3. Paravirtualization.
Hardware virtualization technology. Hypercalls and paravirtualized devices. Microsoft Hyper-V.

Day 4. Hypervisor core.
Virtual Machine Monitor (VMM). CPU & MMU theory. Linux KVM.

## LEVELS & CERTIFICATION

Complexity: intermediate.
Certificate: ZDE HVR (available by request).
Next level training: "Advanced Hypervisor Exploitation".

## TRAINING DETAILS

All modern hypervisor systems share a common foundation of conceptual models: design models, threat models, virtual hardware models – that may be abstracted away from the technical details of a particular implementation. Key principle of this training is to introduce the students to vulnerability research in arbitrary hypervisors through the perspective of their common abstractions, while leveraging implementation-specific technical case studies and the author's many-year experience in operating systems introspection and vulnerability research to facilitate practical applications.

The subject of this training – system internals, vulnerability patterns and trends – represents an essential basis for all offensive and defensive workflows in software security production: such as vulnerability discovery, exploit development, secure systems design, writing safe code and security auditing, as well as ensuring good decisions with 3rd party products selection, deployment and integration in security-critical environments.

The structure of this training is an even mix of theoretical and hands-on experience. Training materials are systematically designed, with each training day focused on one common subsystem of a hypervisor, and divided into four self-contained two-hour blocks that cover relevant theoretical concepts, threat models and attack vectors, known vulnerability patterns and trends, and key technical details of several popular hypervisor implementations.

Exercises in this training are based on suitable real-life tasks and specially crafted with time management techniques in order to enable accelerated learning of skills and internalization of extensive knowledge. Practical environments are based mostly on static source code analysis with occasional binary analysis and debugging labs.

Knowledge presented in this training is based on independent technical research and reverse engineering work done by the author herself, and illustrated with security vulnerabilities discovered by the author, as well as with publicly available sources. Training materials are updated prior to each training session to incorporate important public advancements in the field and new technological developments.

This training is primarily focused on four popular hypervisor implementations: Oracle VirtualBox, VMware Workstation, Microsoft Hyper-V, and Linux KVM. Other virtualization and emulation systems, though not discussed in a dedicated manner, are occasionally featured in specific vulnerability case studies.

## ABOUT THE INSTRUCTOR

Alisa Esage is an independent hacker and founder of Zero Day Engineering. She was the first woman to compete solo at Pwn2Own, where she demonstrated a critical virtual machine escape. She currently works at the cutting edges of the exploit field, between advanced theoretical research and non-trivial exploit engineering for traditionally challenging research targets; such as hypervisors, browsers and basebands.

Alisa is known for her ability to reverse engineer structural patterns behind some of the most mysterious zero-days and architectures, and compress them into practical, high-impact methodologies. Her trainings are used by top-tier researchers, government analysts, and professional red teams worldwide.

## LEARNING OBJECTIVES

Upon completion of this training class the students are expected to have obtained the following knowledge and skills:

- A deeply systematical and generalized perspective on virtualization systems design and internals;
- Recap of the relevant theoretical concepts from operating systems theory, hardware design and application security;
- Familiarity with key implementation-level peculiarities and general design of several major hypervisor products, both open source and proprietary;
- Knowledge of the classes, patterns and trends of software vulnerabilities which are specific to hypervisors, based on deep technical details of several dozens of real-life case studies, and an overview of the publicly exposed vulnerability history;
- Fundamental practical skills of vulnerability discovery and prototyping in large and popular code bases, based on modern toolsets and the author's personal playbook;
- Ability to discover previously unknown vulnerabilities in arbitrary hypervisor software systems;
- Ability to prototype a known hypervisor vulnerability based on a security patch;
- A deep sense of trends and future developments in hypervisor security research.

## HARDWARE & SOFTWARE REQUIREMENTS

All students are required to prepare their own lab and exercise environment for the class.

Hardware and software requirements:

1. Productivity-grade laptop or a desktop computer with a modern HVT-enabled chipset (either Intel VT-x or AMD-V).
2. A modern Linux OS installation, either:
    a. A modern Linux native installation, or
    b. A modern Linux virtual machine with nested virtualization enabled (see Notes).
3. A professional disassembler.
4. An advanced IDE with C/C++ syntax intelligence.

Notes:

- Hands-on labs were tested on Ubuntu Linux 20.04 LTS x64. In case of a different Linux platform flavor, the student would be responsible to specialize the build instructions and introspection guides to it.
- Linux environment will be utilized (among other things) to build VirtualBox and run a virtual machine in the self-build. Unless Linux is running on bare metal, that would require nested virtualization enablement in the base hypervisor.
- Majority of modern desktop hypervisors support HVT pass-through ("nested virtualization") on both Intel and AMD chipsets, which is required for option 2b. Software recommendations for the latter option: VirtualBox, VMware Workstation, Parallels Desktop.

## TRAINING PACKAGES & FEES

This training is available in several formats: live and self-paced, public and private, and various package options.

Online and on-demand (self-paced) trainings are based on a modern streaming platform with high-quality audio and video, and a group chat. The instructor will be available for questions, feedback and technical support, according to the conditions of the specific package.

Payments through the website are subject to merchant fees, that can be waived by paying directly via bank transfer or with crypto currencies.

## LIVE TRAINING

**View-only**

What's included:

- Access to public online training.
- Training slides and materials.
- Training completion certificate.

Price: €3,900.- per person.

**Interactive access**

What's included:

- Everything in the View-only option.
- Feedback and technical support from the instructor in real time during the training.
- Professional examination for the Honors certificate.

Price: €4,200.- per person.

Note: Limited number of seats for the Interactive access option.

## SELF-PACED COURSE

**Basic package**

What's included:

- Video lectures, exercises, and walk-through.
- Training slides and supporting materials.
- Access for 3 months.

Price: €2,500.- NET per person.

**Complete package**

What's included:

- Everything in the Basic package.
- One month of technical support by email.
- One on-demand consultation call with the founder.
- Training completion certificate.
- Lifetime access.

Price: €3,900.- NET per person.

Note: certificates will be available upon request.

## PRIVATE & CUSTOMIZED TRAINING

Minimal private group size is 10 persons. Contact us to check availability.

## LIVE TRAINING DATES & BOOKING

Refer to our website for the dates of the nearest public class.

All our online training courses are offered exclusively at the Zero Day Engineering project (zerodayengineering.com).

Contact us directly for all the bookings and purchases.

# TRAINING PROGRAM

## DAY 1. FOUNDATION

**1.1. Virtualization 101.**

Virtualization technology flavors. Emulation, binary translation, recompilation, raw execution, sandboxing, hardware assisted virtualization. Bare-metal and hosted hypervisors. Hypervisor subsystems: VMM, virtual devices, guest services, hypercall interfaces. Modern trends in development and implementation of hypervisors.

Agenda:
- History of virtualization technology.
- Virtualization technology landscape.
- Terminology and technologies.
- General design model of a hypervisor.
- Threat models, attack surfaces and attack vectors.

Exercise: investigate a popular implementation of a virtualization system in source code.
Target skill: initial orientation in a large source code base.

**1.2. Guest services subsystem.**

Rich virtualization functionality: shared folders, clipboards, smart cards. Drag-and-drop, virtual printer, memory ballooning. Hardware accelerated 3d & 2d graphics and graphic shaders, GPU virtualization, OpenGL and DirectX pass-through. Networking services: DHCP, TFTP, zero-conf, PXE boot. NAT, virtual LAN, inter-VM networking.

Agenda:
- Overview and functions of guest services.
- Relevant OS theory.
- Implementation options.
- Threat models and attack vectors.
- Common security issues.

Exercise: investigate an implementation of guest services in source code.
Target skill: identifying and navigating a specific subsystem in a large code base.

**1.3. Attacking guest services.**

Agenda:
- Threat models and attack vectors.
- Principles of fuzzing and static analysis.
- Examples of known vulnerabilities in guest services.

Exercise: security patch analysis based on source code.
Target skill: identifying and understanding a security vulnerability by analyzing source code modifications.

**1.4. VirtualBox implementation.**

Agenda:
- Overview of design and implementation.
- Guest additions / virtualization tools.
- Building instructions.
- Security research trends.
- Case study: 3d graphics subsystem.

Exercise: find as many bugs as you can in the given attack vector by static analysis.
Target skill: basic vulnerability discovery by pattern-based static analysis in source code.

**Updates 2025 - Lecture 1. Stable Expansion**

This lecture maps to topics of Day 1, plus a general overview of what happened between 2021-2025 in hypervisor security & technology.

- Media outbreaks.
- Review of notable exploits.
- Technology updates.

## DAY 2. VIRTUAL HARDWARE

**2.1. OS & hardware theory.**

CPU privilege modes, the kernel and loadable kernel modules, kernel-userland interfaces. Common classes of computer hardware: processing units, SoCs, buses, peripherals. Fundamental hardware concepts: registers, BARs, I/O, interrupt and transfer management. PIO, MMIO, DMA. FIFO buffers (queues), FILO buffers (stacks), ring buffers, transfer descriptors, interrupts and polling. Device driver functions, skeleton and top-level algorithm.

Agenda:
- OS and hardware ABC.
- Hardware-software boundary and interfaces.
- PCI bus specification.
- USB bus and Host Controller Interfaces.
- Intel 8254xxx (E1000) ethernet controller model.

Lab: live hypervisor introspection.
- Inspecting virtual hardware from within a guest OS.
- Debugging a hypervisor.
- Testing attack vectors.

**2.2. Anatomy of a Linux device driver.**

Common logical blocks of a Linux kernel module. Device driver specifics. *OPS and OS callbacks. Probe function, device initialization and set up, operation routines. Common callback structures: struct pci_driver, usb_driver, drm_driver. Common hardware resource management primitives: ioremap, pci_*, dma_*, and their Linux subsystems.

Agenda:
- Linux kernel module skeleton and building blocks.
- Kernel management hooks.
- Kernel-userland interfaces.
- Resource management primitives.
- Probe function.

Exercise: creating a vulnerability proof-of concept from a binary security patch, part 1: analysis of a virtual device driver in source code.

**2.3. Attacking virtual devices.**

Types of virtual devices: purely emulated devices, paravirtualized devices, pass-through, combined models. Common issues: bugs in PIO/MMIO/DMA handling, bugs in parsing of internal command protocols, unsanitized values in transmit descriptors. Integer and buffer overflows, race conditions, TOCTTOU.

Agenda:
- Threat models and attack vectors.
- Research landscape and trends.
- Vulnerability case studies.

Exercise: creating a vulnerability proof-of concept from a binary security patch, part 2: binary vulnerability patch analysis.
Target skill: effective binary patch analysis of an unfamiliar component in a proprietary code base.

**2.4. VMware Workstation.**

The vmx process, the Backdoor interface, emulated device models. The vmx configuration file, undocumented configuration parameters, useful settings. VMware-specific SVGA and VMCI virtual devices.

Agenda:
- Design and implementation overview.
- Guest additions and virtualization tools.
- Research platform set up instructions and tips.
- Advanced configuration.
- Security research trends.
- Case study: VMware SVGA device.

Exercise: creating a vulnerability proof-of-concept from a binary security patch, part 3: analysis of the virtual device implementation, mapping the vulnerability to software, and contemplating the testcase algorithm.

**Updates 2025 - Lecture 2. Emulated Devices Still**

- Review of Pwn2Own exploits.
- Intel VT-d & VT-c.
- Deep dive: FreeBSD xhci bug (2024).

## DAY 3. PARAVIRTUALIZATION

**3.1. Hardware virtualization technology.**

Overview of Intel VT-x, EPT, and AMD-V virtualization extensions implementations. VMX root operation, VMX non-root operation, VM exit, VM entry, VMCS.

Agenda:
- Intel VT-x: basic concepts.
- Virtual Machine Control Structure (VMCS).
- Programming a hypervisor.
- The hypervisor execution loop.

Exercise: analyze a hardware-assisted hypervisor implementation in source code.
Skill: identifying ultra-specialized low-level functionality in a large code base.

**3.2. Hypercalls & paravirtualization.**

Hypercall interface implementations: legacy – CPU instruction interception, synthetic emulated device; modern - HVT-based. Paravirtualization vs. emulation. Shared memory and synthetic interrupts.

Agenda:
- A legacy hypercall implementation case study.
- A modern hypercall implementation case study.
- Paravirtualization vs. emulation.
- The shared memory paravirtualization interface.
- Para-protocols.

Exercise: investigate a modern HVT-based hypercall implementation in source code of a hypervisor driver and theoretical specifications.

**3.3. Attacking hypercalls & paravirtualization.**

Agenda:
- Threat models and attack vectors.
- Common issues.
- Fuzzing hypercalls.
- Vulnerability case studies.

Exercise: theoretical analysis of a relevant real-life vulnerability and mapping it to hypervisor drivers.
Target skill: deduce a particular vulnerability technical details, practical context and research implications from a brief public mention.

**3.4. Microsoft Hyper-V implementation.**

The hypervisor (hvix64.exe/hvax64.exe), the hypercall interface, the VMBUS. Root partition, child partitions, enlightened and unmodified guest OS, Generation 1 and Generation 2 virtual machines. Emulated and paravirtualized hardware components. Enhanced session mode and the Remote Desktop Services (RDP) infrastructure. VSP/VSC hypervisor driver model, Linux Integration Services and Microsoft Windows Hyper-V drivers. VBS (Virtualization Based Security), WMI (scripting), the Hypervisor API.

Agenda:
- Design and implementation overview.
- Attack surfaces and attack vectors.
- Attack surface reduction in practice.
- Research platform set up.
- Security research trends and tips.
- Case study: Virtual Network Switch.

Exercise: analyze a paravirtualized device driver for purposes of fuzzing.
Target skill: conceptual modeling of a custom specialized fuzzer implementation.

**Updates 2025 - Lecture 3. Hypervisor Fuzzing, Anyone?**

- Massive updates to hypervisor fuzzing in 2021-2025. Trends.
- Review of academic literature and proof-of-concept implementations.
- Morphuzz, HyperPill, IRIS, V-Shuttle, VD-Guard, Nyx, Truman, HyperFuzzer.

## DAY 4. VMM

**4.1. CPU & MMU theory.**

CPU ISA basics, interrupts and exceptions. Privileged and unprivileged CPU execution modes. Special instructions with respect to virtualization requirements. Conditional and unconditional VM-exits. x86_64: CPUID, GETSEC, INVD, XSETBV. MSR and CRx registers. Pipelines, prefetching, and optimization. MMU hardware operation overview. Paging modes: 32-bit, PAE, 4-level. PML4E, PDPTE, PDE, PTE.

Agenda:
- CPU design and operation software theory.
- Memory address translation process.
- OS kernel management of CPU & MMU hardware operation.

Exercise: theoretical analysis of a VM exit and CPU instruction emulation in source code.

**4.2. Virtual Machine Monitor subsystem.**

CPU & MMU virtualization in practice. Full CPU emulation, special instruction emulation, partial pass-through of instruction functionality, CPU instruction interception. HVT, binary translation and interception-based handling of special instructions in practice. Virtual MMU: shadow page tables, nested page tables and hardware-assisted MMU virtualization technology (EPT).

- Legacy (software-based) and modern (hardware-assisted) CPU virtualization techniques.
- Legacy and modern MMU virtualization techniques.
- Common VMM designs in practice.

Exercise: theoretical analysis of an MMU virtualization implementation in source code.

**4.3. Attacking the VMM.**

Common attack vectors: CPU instruction emulation, VM exit handling, nested VMX emulation. Common classes of issues: design issues due to mishandled hardware specification; memory corruptions in instruction emulation; memory page table updates and shadowing sanitization issues. Speculative execution bugs.

Agenda:
- Threat models and attack vectors.
- Offensive research trends.
- Vulnerability case studies.

Exercise: analysis of a speculative execution bug and exploit.
Skill: rapid grasping of a low-level attack concept; paradigm shift from vulnerability discovery to exploit engineering.

**4.4. Linux KVM implementation.**

Agenda:
- Design and implementation overview.
- KVM API and notable consumers.
- Security research trends.
- Implementation deep dive.

Exercise: create a proof-of-concept testcase for a VM-escape vulnerability based on a security patch in source code, and crash VirtualBox.

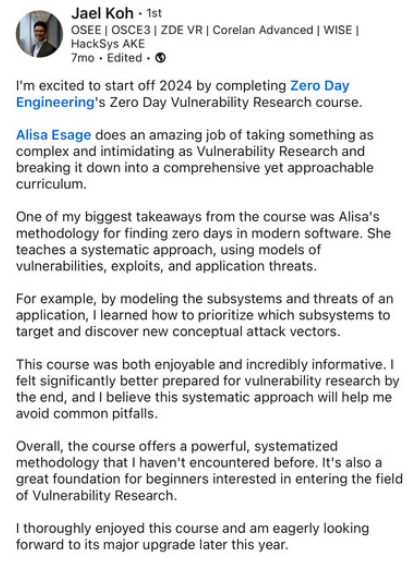
Note**:** the training agenda may experience minor changes that won't impact expected outcomes of the training.

**Updates 2025 - Lecture 4. The Speculative Threat Is Not Speculative.**

Focus on new developments which map to Day 4 VMM and hardware topics. Emergent technology and no-longer speculative exploits.

- Review of side channel and speculative execution attacks on hypervisors.
- AMD SEV-SNM.
- Intel TDX.

# INDUSTRY FEEDBACK - ZERO DAY ENGINEERING TRAININGS



**Jael Koh** · 1st
OSEE | OSCE3 | ZDE VR | Corelan Advanced | WISE | HackSys AKE
7mo · Edited ·

I'm excited to start off 2024 by completing Zero Day Engineering's Zero Day Vulnerability Research course.

Alisa Esage does an amazing job of taking something as complex and intimidating as Vulnerability Research and breaking it down into a comprehensive yet approachable curriculum.

One of my biggest takeaways from the course was Alisa's methodology for finding zero days in modern software. She teaches a systematic approach, using models of vulnerabilities, exploits, and application threats.

For example, by modeling the subsystems and threats of an application, I learned how to prioritize which subsystems to target and discover new conceptual attack vectors.

This course was both enjoyable and incredibly informative. I felt significantly better prepared for vulnerability research by the end, and I believe this systematic approach will help me avoid common pitfalls.

Overall, the course offers a powerful, systematized methodology that I haven't encountered before. It's also a great foundation for beginners interested in entering the field of Vulnerability Research.

I thoroughly enjoyed this course and am eagerly looking forward to its major upgrade later this year.

**Peleg Hadar**
@peleghd

The Hypervisor Exploitation One training by @alisaesage was simply incredible. Alisa is a great teacher with a lot of experience in the Vuln research field.
The Training contains practical exercises, it provides the knowledge you'll need for starting a Hypervisor vuln research!

**Mohammad Hussam Alzeyyat** · 2nd
Vulnerability Researcher/Cyber Security(Instructor,Labs Developer,Upwork Top Rated)/OSED,OSMR
2mo · Edited ·

I just finished the "Masterclass: Hacking open source fuzzers for smarter bughunting" from zerodayengineering.com by Alisa Esage.

The 4-hour training is an eye-opening to more advanced fuzzing and it's more oriented to learn how to customize the fuzzer for your own purpose and not to learn how to fuzz.

The masterclass focuses on AFL and WinAFL since those are the most powerful frameworks for fuzzing and explains the anatomy of fuzzing and the differences between the old generation fuzzers methodology and the new ones.

I don't think the masterclass is for beginners as much as for intermediate people or those with fuzzing experience.

**Josh Pitts**
@ausernamedjosh

The JavaScript Engines for Hackers by @zerodaytraining was great. Shortened the time to get familiar with V8. Thx @alisaesage!

**Gal Z**
@0xgalz

What a blast! Just finished @alisaesage 3 days deep technical Hypervisor Vulnerability Research course. Incredible and well designed with high quality materials.
Thanks for everything! Now, off to look at some VMM code \o/

**Carlomagno A.** · 2nd
Security Researcher
6mo ·

My review for #ZeroDayVulnerability course by Alisa Esage.

Alisa's does an amazing job at explaining more than decade of wisdom, insights and knowledge on a complex topic as Vulnerability Research in a 4 days course. The value of this information and training goes beyond just specific attack vectors or simplistic how-to's, it covers deep technical hands on training on real world vulnerable applications.

The segment of the course that I enjoyed the most was the establishment of the correct/appropiate mindset, definition of abstract models/frameworks, cognitive skills needed to better improve, this specific points matched with a well structured self-study roadmap will allow you to track your progress by establishing a feedback loop on the concepts that you need to improve upon to excel at Vulnerability Research.

**rui**
@fdiskyou

Alisa's training completely changed my mindset and the way I was looking at hypervisors. I highly recommend the Advanced Hypervisor Exploitation training. Actually, if you're interested in hypervisors it's mandatory.

**Kapil Khot** · 1st
Security Consultant/Penetration Testing | CoreLAN Exploit Dev | OSCP
7mo · Edited ·

Here's my review for #ZeroDayVulnerabilityResearch class offered by Zero Day Engineering. It's an intensive four days training where Alisa Esage shares some important tips from her Pwn2Own competition and decade long zero day research experience.

She walked us through real world application's code base to demonstrate how to indentify its subsystems and which one to prioritze for testing. Additionally, she walked us through some of the real world vulnerable code snippets from various enterpriese and real-world applications to demonstrate several bug classes such as memory corruption vulnerabilities, hardware bugs, and logic bugs etc. Furthermore, she also touched base on the security weaknesses of one of the memory safe languages and shared research tips on upcoming technologies.
One of the modules, Fuzzing Masterclass took us through the internals of coverage guided fuzzers and offered guidance on customizing them for applications not supported out of the box.

Overall, this course helps you build a zero day research mindset, guides you on what needs to be done, and forces you to figure out the most of the "how-to" part, which in my opinion is the best thing.

**Mario Romero Serrano** · 2nd
Senior Vulnerability Researcher - Cryptography Research Centre at Technology Innovation Institute (TII) | Cryptograp...
5mo ·

The last few months I have been enjoying and learning about vulnerability research from one of the best trainings available today.

The 4-day Zero Day Vulnerability Research course from Zero Day Engineering by Alisa Esage is not only exceptional on a technical level, but it also teaches you a methodology to follow, and personally, what for me is most valuable and concentrates Alisa's years of experience and knowledge: it introduces you to the mindset necessary for your vulnerability research to be successful.

I am already applying the knowledge I have acquired and I am eager to continue learning from her. I'm sure I'll be taking some more of her courses. For now, I will enjoy the course for the second time.

Don't miss the opportunity to do it:

**Jacolon Walker**
@call_eax

Big thanks to @alisaesage and @zerodaytraining for their top-notch training content! Just wrapped up a refreshing course with promising results - from bug discovery to exploitable vulnerability in just 9 days. Dive into my chronicle here: https://t.co/PCfqzCLs6B

**Veer Singh** · 1st
Security Practitioner
7mo ·

I recently (2023) completed the "Hypervisor Vulnerability Research" course by Alisa Esage, which provided a comprehensive exploration of virtualization technology and its security aspects. The course began with a clear introduction to virtualization technology, distinguishing between various types of hypervisors. We then explored the rich functionality of guest services, understanding their practical implementation and potential security challenges. The module on attacking guest services equipped us with valuable perspective on attack vectors, and the in-depth look at VirtualBox's implementation was insightful. Overall, this course is highly recommended for both beginners and experienced professionals looking to enhance their virtualization security skills. Here is a link to the course for anyone interested -
https://lnkd.in/g5N2gNmT

**Training: Hypervisor Vulnerability Research**
zerodayengineering.com

**scryh**
@scryh_

Just finished the @zerodaytraining hypervisor vulnerability research offline course by @alisaesage. Impressive research bundled into very well structured lessons, which are communicated in a clear and comprehensible way. Learned a lot. Thank you very much!
https://t.co/iW3UAGBtUq

## APPLICATIONS & INQUIRIES

E-mail: contact@zerodayengineering.com.
Public cohorts and booking: zerodayengineering.com
Socials: @zerodaytraining